Design Portfolio

Oliver Buehrer

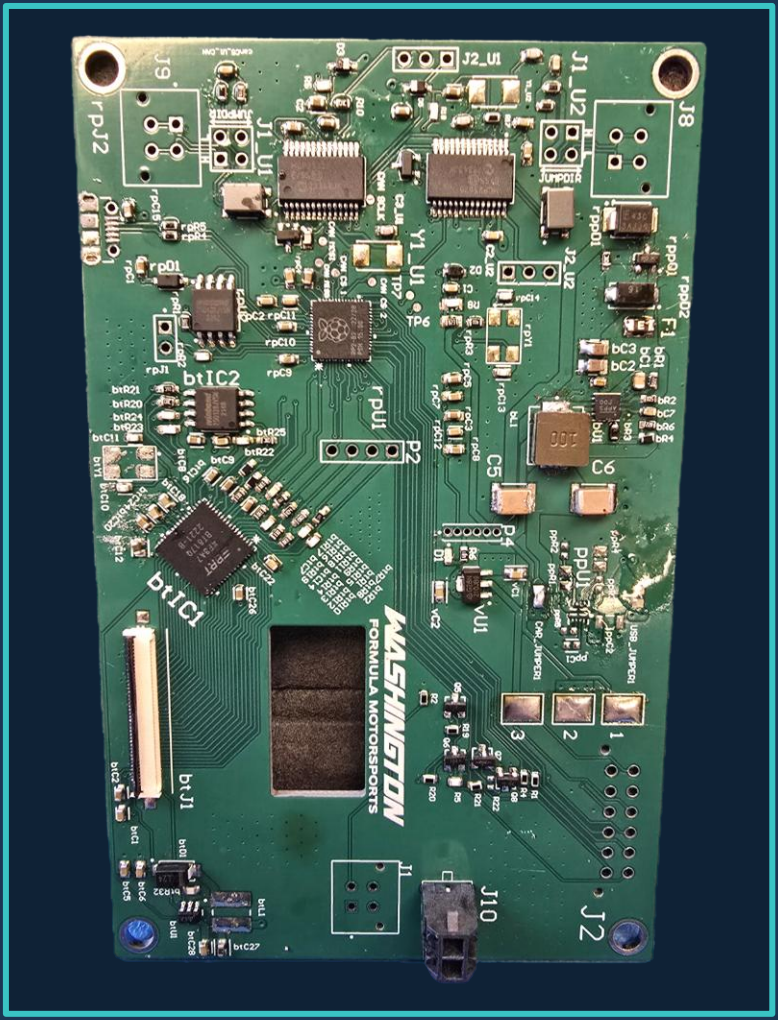
linkedin.com/in/oliver-buehrer/

UWFM

University of Washington Formula Motorsports

CHARGING CART PCB

The Charging Cart PCB is a custom 4-layer printed circuit board, designed for UWFM's electric racing vehicle charging station. The board manages charge-state display output, dual CAN bus communication, and operator input via rotary encoders.



ROLE

Sole PCB designer and embedded systems engineer. Responsible for schematic design, component selection, PCB layout, and firmware bring-up.

KEY SPECS

- 4-layer PCB (Signal / GND / PWR / Signal)
- RP2040 microcontroller (dual-core ARM Cortex-M0+)
- 2× MCP25625 CAN-to-SPI controllers
- BT817Q-T SPI graphics controller
- Bourns PEC11R quadrature encoders

4
PCB LAYERS

2×
CAN BUS

UWFM

University of Washington Formula Motorsports

HARDWARE OVERVIEW

MICROCONTROLLER

RP2040 — dual-core ARM Cortex-M0+ @ 133 MHz, 264 KB SRAM, USB, SPI, I2C

DISPLAY CONTROLLER

BT817Q-T — SPI-driven GPU for EVE-based TFT displays

CAN CONTROLLERS

2× MCP25625 — integrated CAN controller + transceiver, SPI interface

USER INPUT

Bourns PEC11R — quadrature rotary encoders, 5V outputs

POWER SUPPLY

5V DC input → 3.3V LDO regulator for MCU and logic

LEVEL SHIFTING

2N7002 signal MOSFETs — 5V encoder outputs → 3.3V GPIO

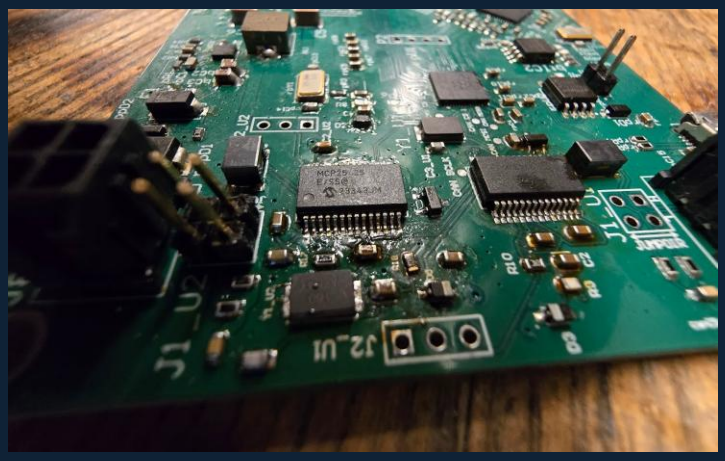
UWFM

University of Washington Formula Motorsports

DESIGN HIGHLIGHTS

DUAL CAN BUS

Two MCP25625 CAN-to-SPI controllers share the RP2040 SPI bus with independent chip selects. Enables simultaneous connection to separate vehicle CAN networks (e.g., BMS and charger).

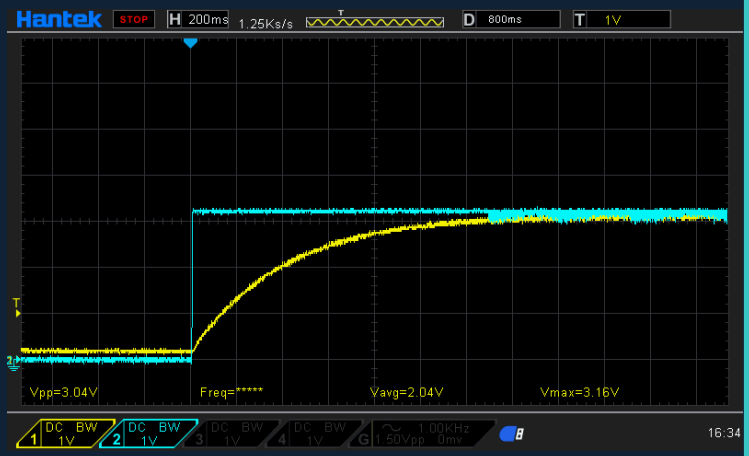


ENCODER LEVEL-SHIFTING

2N7002 N-channel MOSFETs shift 5V encoder outputs down to 3.3V GPIO-safe levels. Drain pulled to 3.3V via resistor; gate driven directly by encoder output pin.

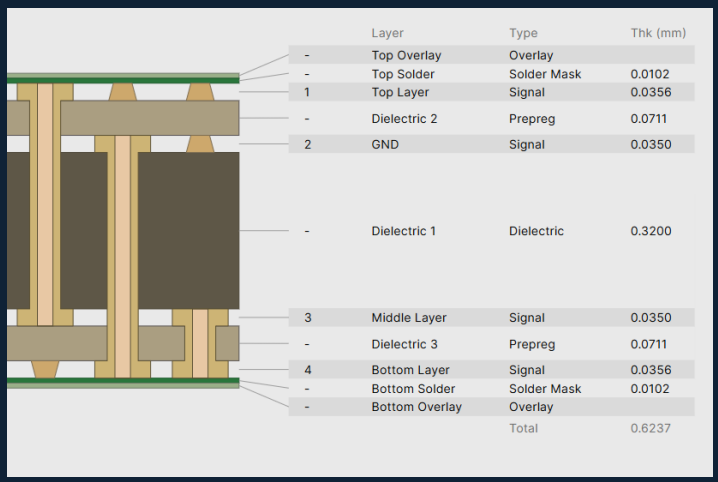
RC RESET CIRCUIT

RC network on the MCU RESET pin generates a clean power-on-reset delay. Capacitor holds reset low until supply is stable, ensuring reliable MCU startup without an external supervisor IC.



4-LAYER STACKUP

Signal / Ground plane / Power plane / Signal stackup reduces EMI and provides low-impedance power distribution. Dedicated ground and power planes eliminate the need for extensive decoupling trace routing.

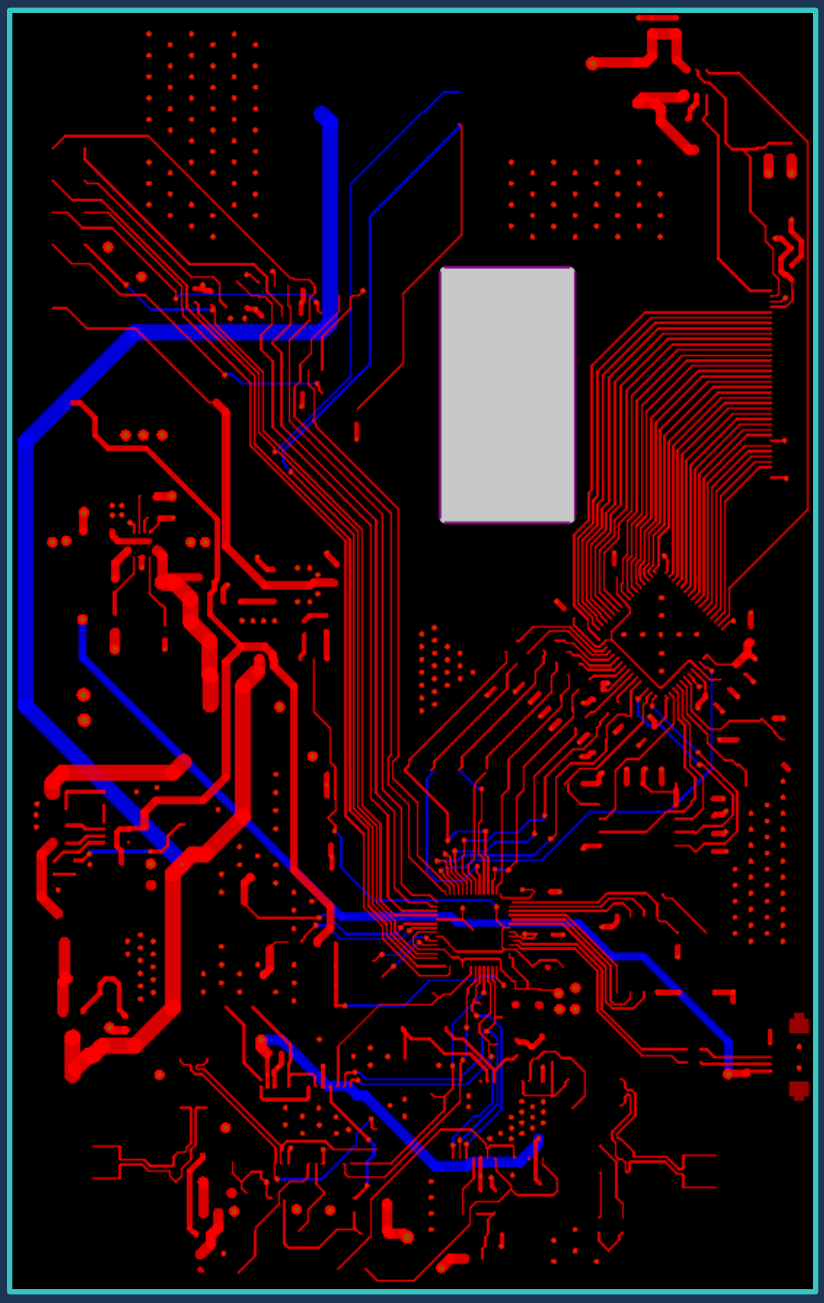


	Layer	Type	Thk (mm)
-	Top Overlay	Overlay	
-	Top Solder	Solder Mask	0.0102
1	Top Layer	Signal	0.0356
-	Dielectric 2	Prepreg	0.0711
2	GND	Signal	0.0350
-	Dielectric 1	Dielectric	0.3200
3	Middle Layer	Signal	0.0350
-	Dielectric 3	Prepreg	0.0711
4	Bottom Layer	Signal	0.0356
-	Bottom Solder	Solder Mask	0.0102
-	Bottom Overlay	Overlay	
		Total	0.6237

UWFM

University of Washington Formula Motorsports

PCB LAYOUT & VALIDATION



BRING-UP CHECKLIST

- ✓ PCB designed and manufactured (4-layer)
- ✓ DRC / ERC errors resolved
- ✓ Power rails verified (5V, 3.3V)
- ✓ GPIO / actuator I/O verified
- ✓ Encoder level-shift verified on scope
- ▶ CAN bus bring-up *[in progress]*
- ▶ Graphics controller init *[in progress]*
- ▶ Full firmware integration *[in progress]*

✓ = completed ▶ = in progress

DESIGN SPECS

Layers: 4-layer FR4 (Sig / GND / PWR / Sig)

Tool: Altium Designer (schematic + layout)

Key ICs: RP2040, BT817Q-T, 2× MCP25625, MPQ4423

Power: 24V / USB 5V input → 5V buck → 3.3V LDO

Sheets: 10-sheet hierarchical schematic

Role: Sole designer — schematic, layout, firmware

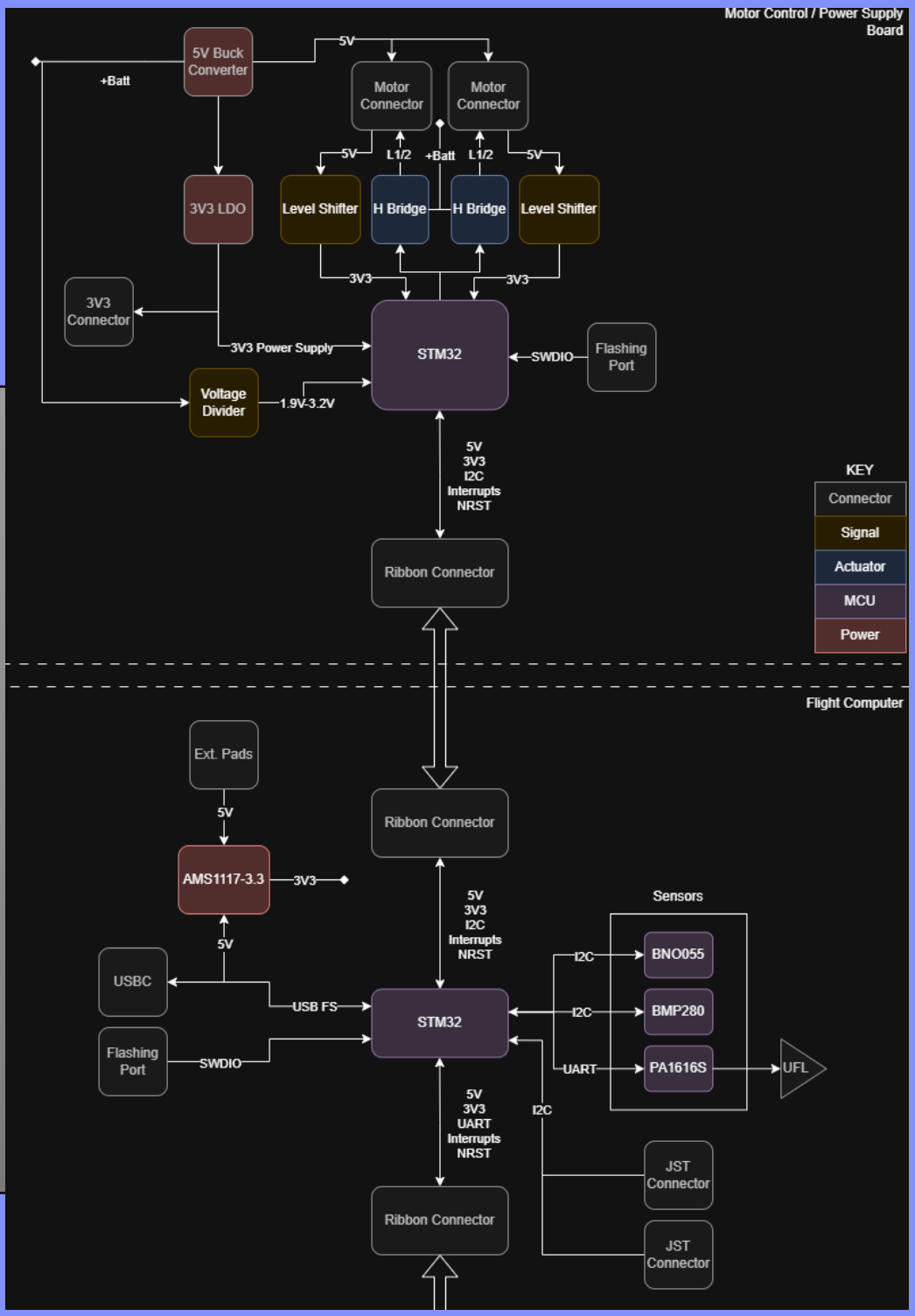
SARP

Society for Advanced Rocket Propulsion at the University of Washington

HARDWARE OVERVIEW

ARES is an autonomous recovery system controlled by a stack of 2 PCBs. It is guided by two custom Black Pills with the STM32 F401CCU6 chip.

As the Responsible Engineer for the controls and electronics of the system, I have personally written the firmware for the MCPS as well large revisions of the MCPS hardware.



System Overview

The MCPS steps down 12V from the motor through a 5V buck converter and a 3.3V linear drop-out regulator. It sends pulse width modulation signals to drive the motor's H-bridges, which it computes based on steering inputs from the flight computer.

The MCPS uses a ribbon connector to provide the flight computer with 3.3V power, as well as data lines to send and receive data via i2c.

The flight computer has an imu, altimeter, and GPS which are used to localize. The flight computer uses this to decide steering deflections to send the MCPS.

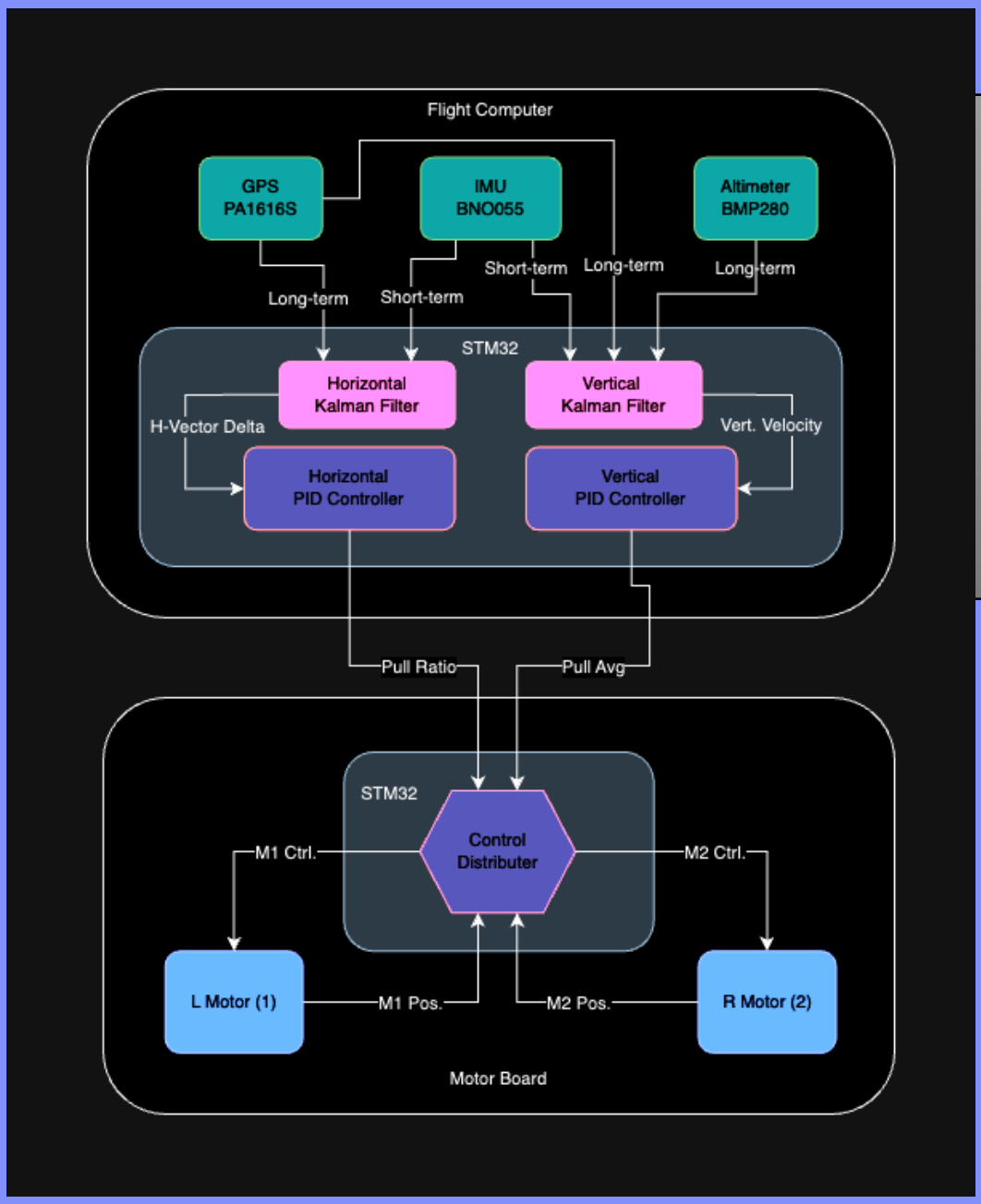
SARP

Society for Advanced Rocket Propulsion at the University of Washington

FIRMWARE OVERVIEW

ARES control system overview

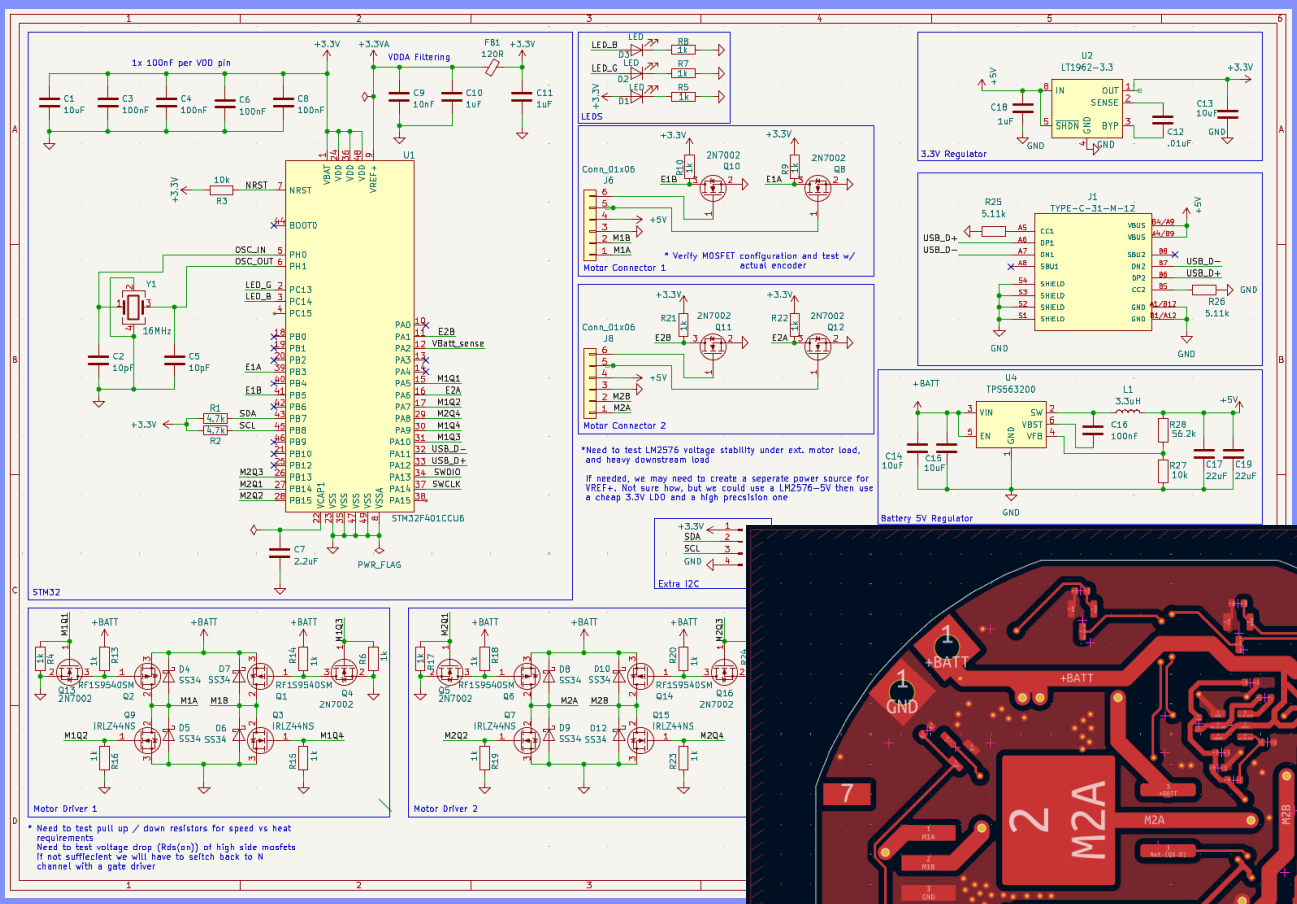
The flight computer uses sensor input through a vertical (drop faster vs slower) and horizontal (left/right) Kalman filters to create steering inputs for MCPS.

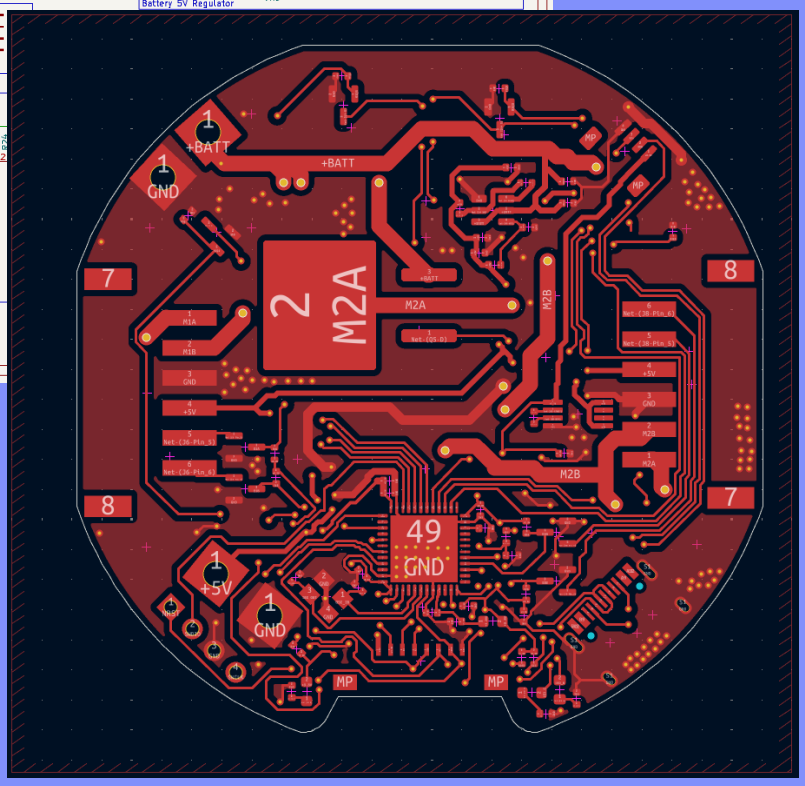


The MCPS takes this steering input, along with motor encoder signals, to calculate and send power to each motor.

SARP

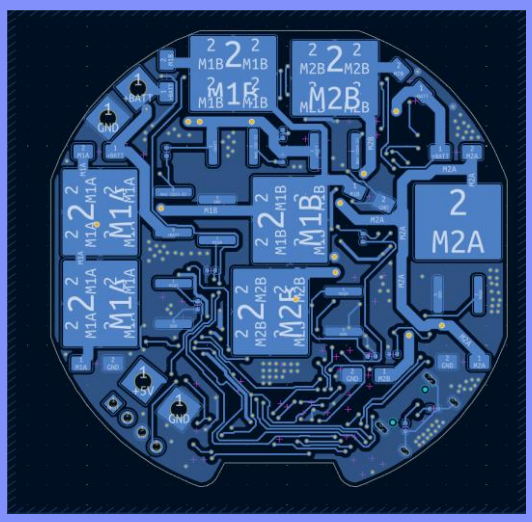
Society for Advanced Rocket Propulsion at the University of Washington





I redesigned the MCPS including a schematic overhaul and re-routing:

- Changed the power delivery system to a 5V buck converter and a 3.3V linear drop-out regulator.
- Modified encoder step-down circuitry to use a signal MOSFET to pull GPIO high.
- Fixed pinouts which caused problems with firmware.
- Fixed rushed DRC (layout design rule check) errors with previous iteration in component spacing, resulting in layout re-structure.



SARP

Society for Advanced Rocket Propulsion at the University of Washington

MCPS Github Repository
https://github.com/SARP-ARES/MCP_Firmware

The MCPS firmware is written in C++ and organized by classes including the Motor control, motor PID controller, and an input distributor.

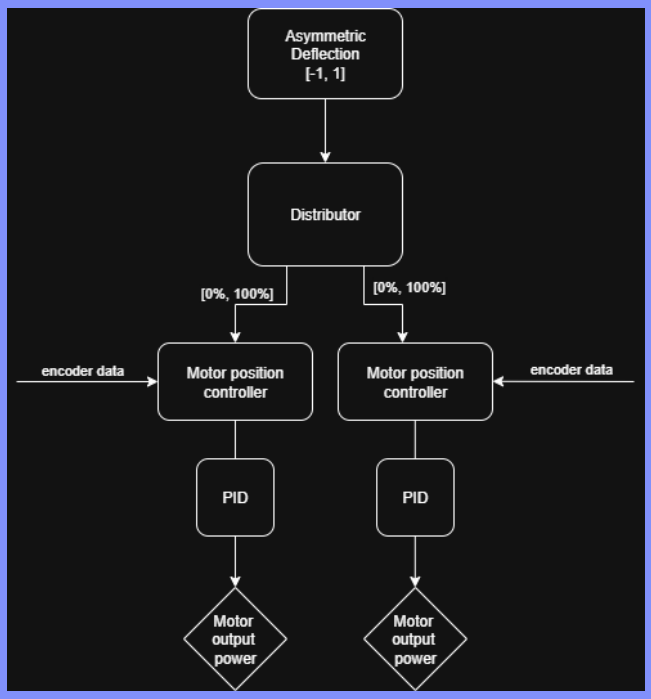
```cpp
// Class which controls DC motor with ethan's mosftet motor board
#ifndef _MOTOR_H_
#define _MOTOR_H_


#include "mbed.h"
#include "PID.h"

class Motor {
    private:
        // Defining constants from arduino
        const int LOW = 0;
        const int HIGH = 1;

        DigitalOut motorPin1;
        DigitalOut motorPin3;
        PwmOut motorPin2;
        PwmOut motorPin4;
        InterruptIn encoderA;
        InterruptIn encoderB;

        // Encoder state boolean
        bool aUp = false;
        bool bUp = false;

        const float PI = 3.1415926535;

        volatile int encoderCounter = 0; // Counter for the encoder tick
        const int countsPerRev = 64;
        const float gearRatio = 150; // Might need to be updated for dif
        const float spoolDiameter = 0.5; // Spool Diameter IN INCHES. Si
        const float MAX_DEFLECTION = 36; // INCHES

        const float totalCounts = countsPerRev * gearRatio; // Total cou
        long position;
        float angle;
        float rotations;
        bool powerPositive;

        PID motorPID;


        void aRiseCallback();
        void bRiseCallback();
        void aFallCallback();
        void bFallCallback();


    public:

        void updateGlobals();

        Motor(PinName PIN_A, PinName PIN_B, PinName MOTOR_1, PinName MOT
                PinName MOTOR_4, const PID& pidObject);
        void motorPower(float power);
        void spinDegrees(int degrees);
        int getDegrees();
        long getDisplacement();\
        void lineTo(float retraction, int delay);

};

#endif //_MOTOR_H_
```

Motor class header file

My third iteration of encoder control code based off callback functions. Previous iterations did not track ticks consistently through direction changes.

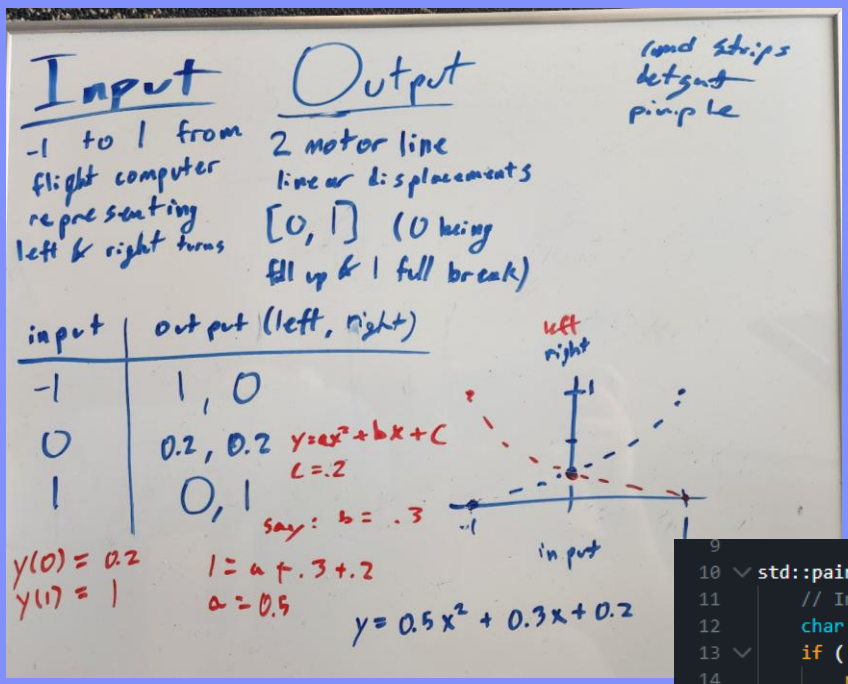
```cpp
void Motor::bRiseCallback() {
    bUp = true;

    if (aUp) {
        encoderCounter++;
    } else {
        encoderCounter--;
    }
}

void Motor::aFallCallback() {
    aUp = false;

    if (bUp) {
        encoderCounter++;
    } else {
        encoderCounter--;
    }
}

void Motor::bFallCallback() {
    bUp = false;

    if (!aUp) {
        encoderCounter++;
    } else {
        encoderCounter--;
    }
}
```

Motor control system overview



SARP

Society for Advanced Rocket Propulsion at the University of Washington



Distributor class takes a char array pointer over I2C serial and outputs control flap open percentages for the controllable rogallo-wing style reserve parachute.

```
std::pair<float, float> Distributor::getMotorOutputs()  {
    // Integer from -100 to 100
    char buf[DISTRIBUTOR_BUFFER_SIZE];
    if ( !ser->readline(buf, DISTRIBUTOR_BUFFER_SIZE) ) {
        return std::make_pair(NAN, 0);
    }
    int input;
    int res = sscanf(buf, "%d", &input);
    if (res != 1) {
        return std::make_pair(NAN, 1);
    }


    // Buf processing
    int digit1 = 100*( buf[1] - '0' );
    int digit2 = 10*( buf[2] - '0' );
    int digit3 = buf[3] - '0';
    int sum = digit1 + digit2 + digit3;
    if (buf[0] == '-') {sum*=-1;}

    printf("Distributor turn angle: %d\n", sum);


    float ctrl = ( (float) sum ) / 100.0;
    float leftPull = 0.5 * (-ctrl) + 0.5;
    float rightPull = 0.5 * ctrl + 0.5;

    return std::make_pair(leftPull, rightPull);
}
```

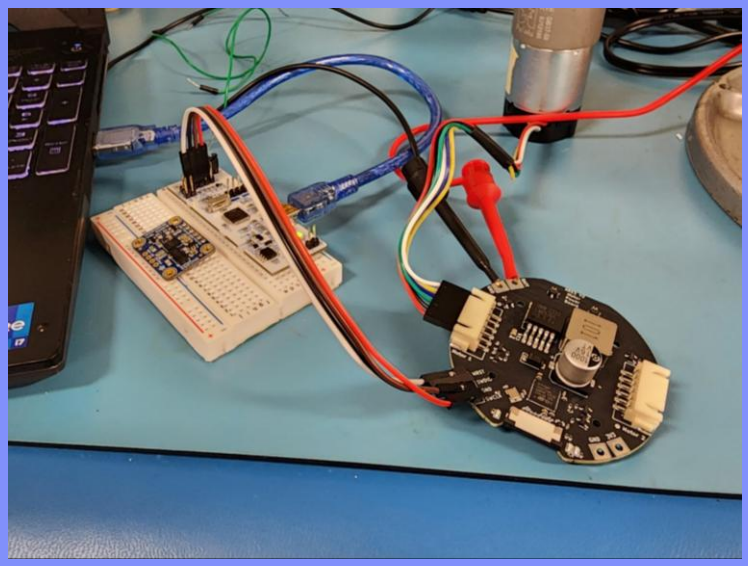
These extension percentages get passed into the motor's controls which uses the PID to turn the motors to the appropriate position.

```
    while (true) {
        ThisThread::sleep_for(10ms);

        std::pair<float, float> spoolExtensions = distributor.getMotorOutputs();

        // if the first float is a NAN, keep each extension value the same4
        leftExtension = (isnan(spoolExtensions.first)) ? leftExtension : spoolExtensions.first;
        rightExtension = (isnan(spoolExtensions.first)) ? rightExtension: spoolExtensions.second;

        motor1.lineTo(leftExtension, 10);
        motor2.lineTo(rightExtension, 10);

        printf("main loop\n");
    }
}
```

SARP

Society for Advanced Rocket Propulsion at the University of Washington

prototype H bridge boards (above, center), as well as the MCPS board (below)



Testing includes iterating, pushing code, observing behavior. The goal is to show hardware and firmware work together. Sometimes this means gathering data (right).

Every day or feature fix I push to github repository to ensure documentation and code history.

2/26
Flip test success:

Reboot stm test	
Pos	Neg
11	9

Power supply on/off	Did it switch?
Yes	No
0	5

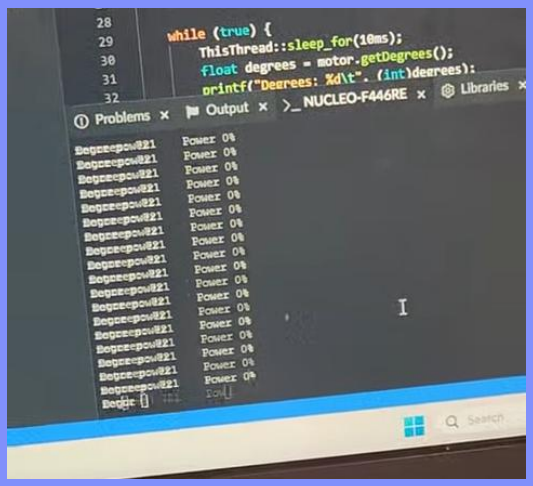
Motor polarity switch	Did it switch?
Yes	No
5	0

Reboot stm with delay	Did it switch?
Yes	No
6	2

Off with longer delay	
Pos	Neg
4	3

Every firmware change and hardware setup is prone to many different points of failure. Some debug strategies I use often are:

- serial feedback (only worked with test board)
- Motor and LED feedback
- multimeter continuity tests
- signal testing with an oscilloscope



PID testing feedback

Core Hardware Engineer for FTC 6220

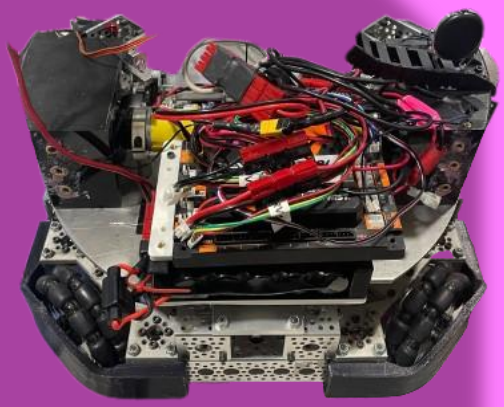
Specialization in 3D printed hardware

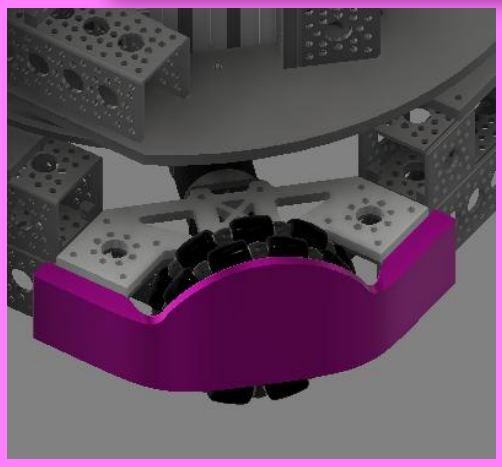
3D Printing material mechanics lead
Project manager, ran team project schedules

Robot Main Shell

Objectives met:

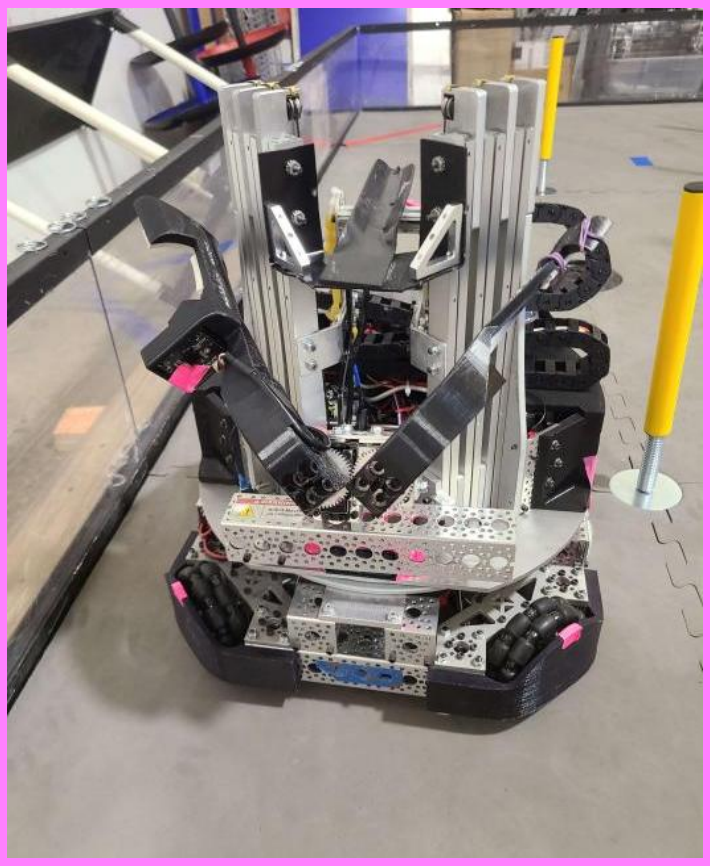
- Impact resistant (Taulman Alloy 910 Nylon)
- Accessible battery (sliding dove tail)
- 2-minute removal for electronic maintenance
- Protected main power switch
- Ease of printing (no supports)





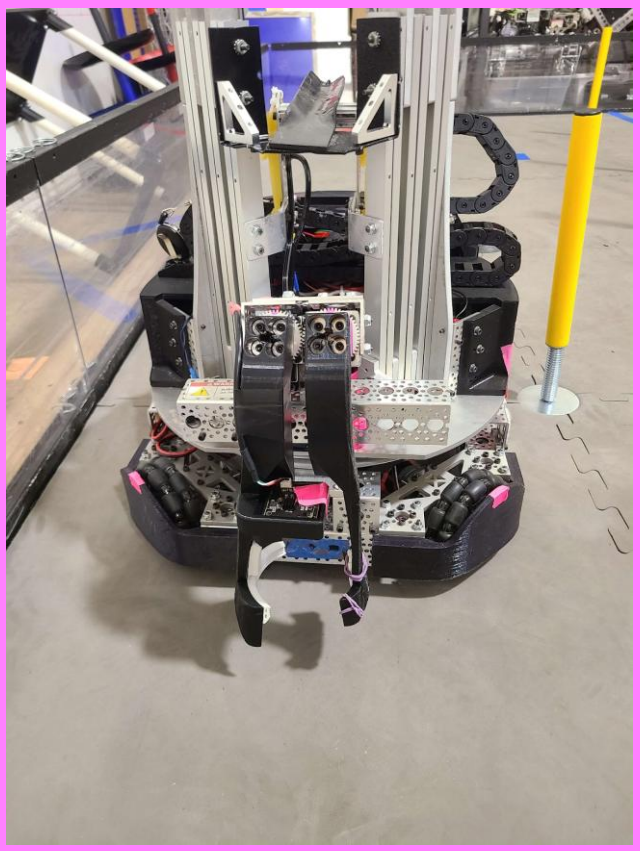
Wheel Guards

- Impact resistance (Taulman Alloy 910 Nylon)
- Profile allows robot to rotate and bounce off objects
- Ease of installation (embedded square nuts + screw-tapped nylon = nylock nuts)

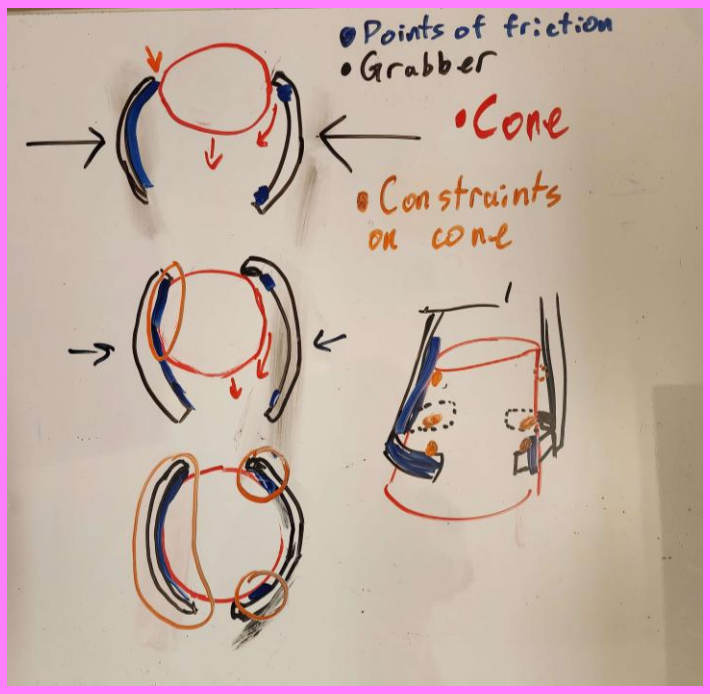


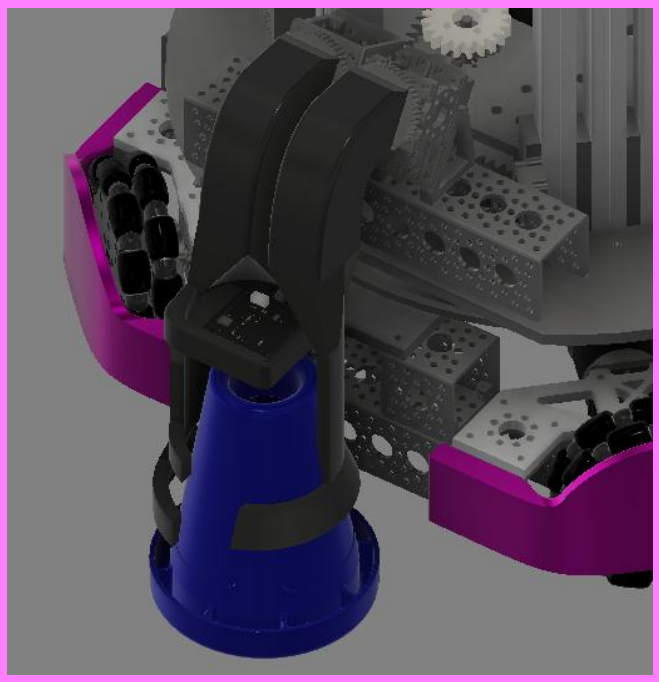
“Butterfly” Retracting Claw

Designed 5+ iterations for a claw mounted at a 60-degree angle to retract inside the silhouette of the robot when fully open. This made a wide grab-area but introduced a mechanical disadvantage.



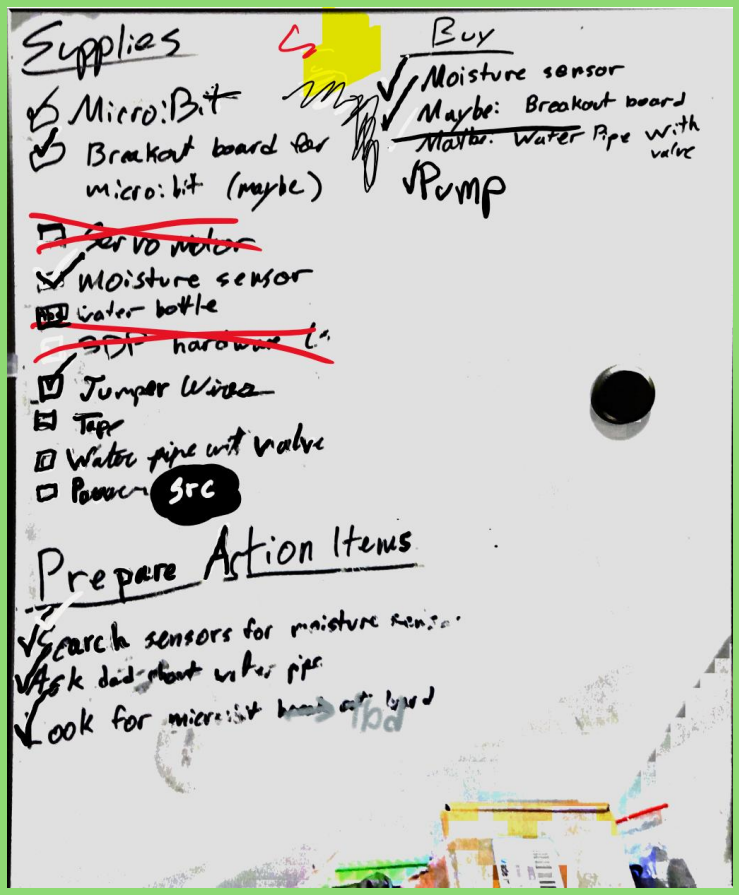
To hold securely with low force, I intentionally designed points of friction to constrain the cone in only one position. Until it reaches that, the cone will get pulled into the center





Micro:Bit Plant Water Automation

Repurposing an old micro:bit micro controller to make a simple control loop to keep my plant watered. The micro:bit uses passthrough power through a breakout board to power a 9V DC vane pump. It provides user feedback and debug values through 5x5 led display.

Moisture sensor Values

Air	1023
2/3 submerged in water	676
Wet	1023
Dried but damp?	1023
Dry dirt	1023
Recently watered dirt	500-450